

Encoding and Decoding Touch-Tone (DTMF) Signals and FIR Filter Design GUI

Group 7: Ajayvarman, Dakyung, Sangeun, Siddharth

Background

The DTMF (Dual Tone Multi-Frequency) lab applies signal processing techniques to simulate and decode telephone touch-tone signals. These signals are generated by combining two sinusoidal tones corresponding to a keypad row and column frequencies. The lab focuses on designing and implementing bandpass FIR filters to isolate DTMF tones and automatically decode them. This real-world application highlights how filters are used to separate and analyze frequency components in a composite signal.

The FIR Filter Design lab explores methods for designing practical Finite Impulse Response (FIR) filters using MATLAB. FIR filters are essential in digital signal processing for their ability to approximate ideal frequency responses with finite coefficients. This lab introduces windowing techniques, such as the Rectangular and Hamming windows, and demonstrates their impact on filter performance, including ripple and transition width. The purpose of the lab is to design lowpass filters, analyze their frequency responses, and understand the trade-offs between filter sharpness and ripple characteristics. It also involves using MATLAB tools like `freqz` for evaluating frequency responses and measuring design accuracy. At the end of the project, the simple application for Low-pass Filter designer, which receives the sampling frequency, cutoff frequency, filter order and the window type as its parameter.

Simple Bandpass Filter Design

Filter Design Specifications:

For each of the eight bandpass filters, choose L such that only one DTMF frequency lies within the passband, and all other frequencies lie in the stopband

Filter Length L:

The larger the value of L, the narrower the filter's passband becomes, which can lead to better separation of frequencies

However, using larger values of L requires more computation and may introduce additional challenges in meeting the DTMF specifications

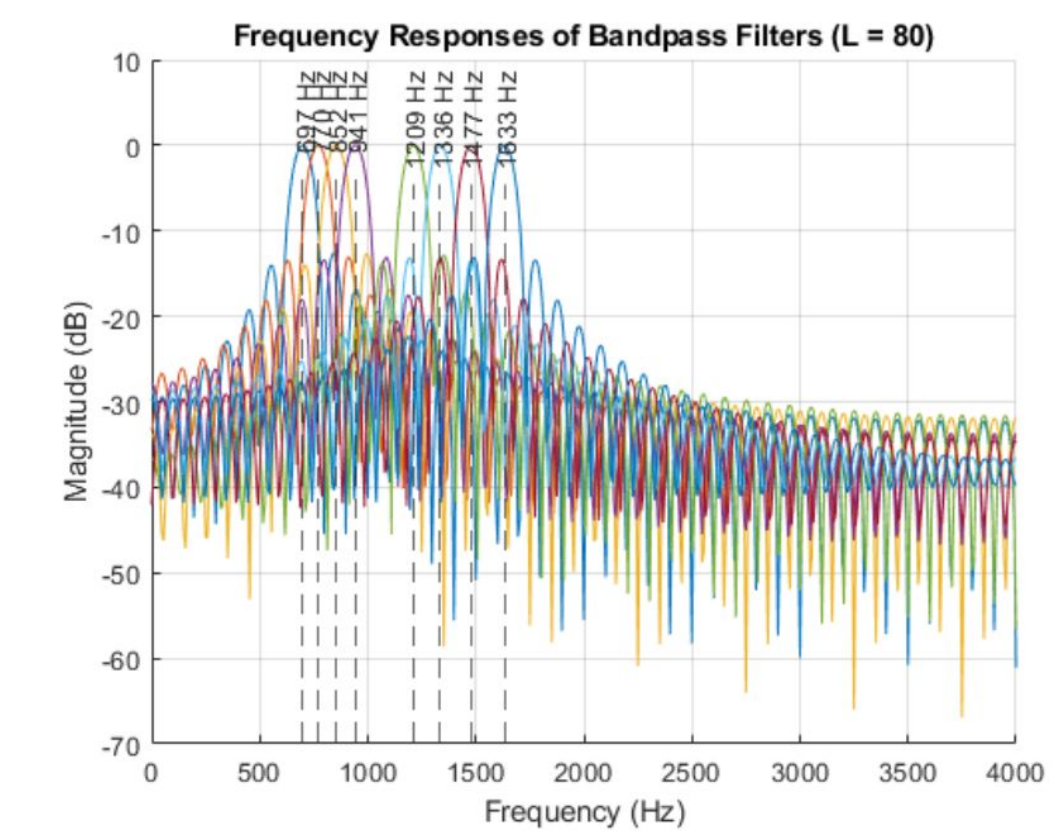
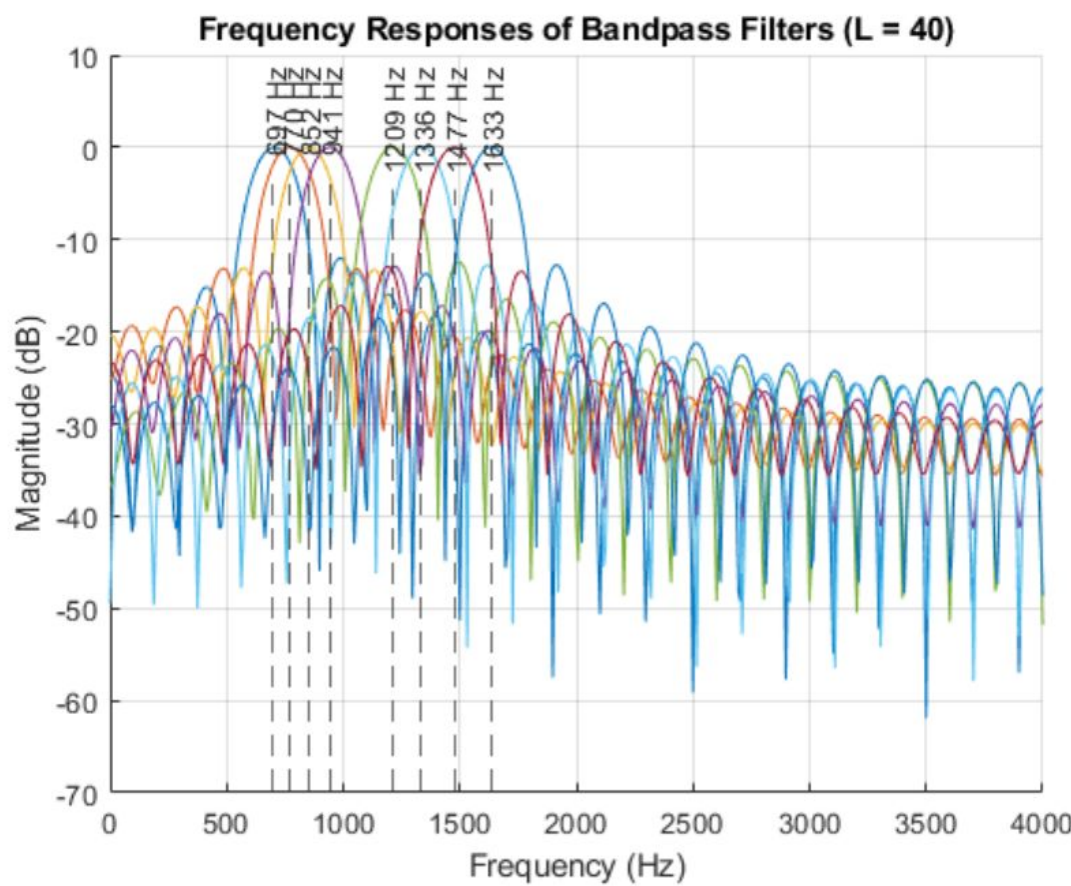
• For L=40:

The passbands will be wider and may not isolate the DTMF frequencies as well, leading to potential overlap between the DTMF components

• For L=80:

The passbands will be narrower, leading to better isolation of the individual DTMF frequencies

This would likely be a better choice for ensuring that only one frequency lies in the passband, and the others are well-rejected



A Scoring Function

Signal Normalization

The input DTMF signal (`xx`) needs to be normalized so that its values lie within the range of [-2,+2]

The amplitude of the two sinusoids (which make up the DTMF signal) is approximately 1.0

Scoring Rule

The maximum amplitude of the output (`|y[n]|`) is compared to a threshold of 0.59

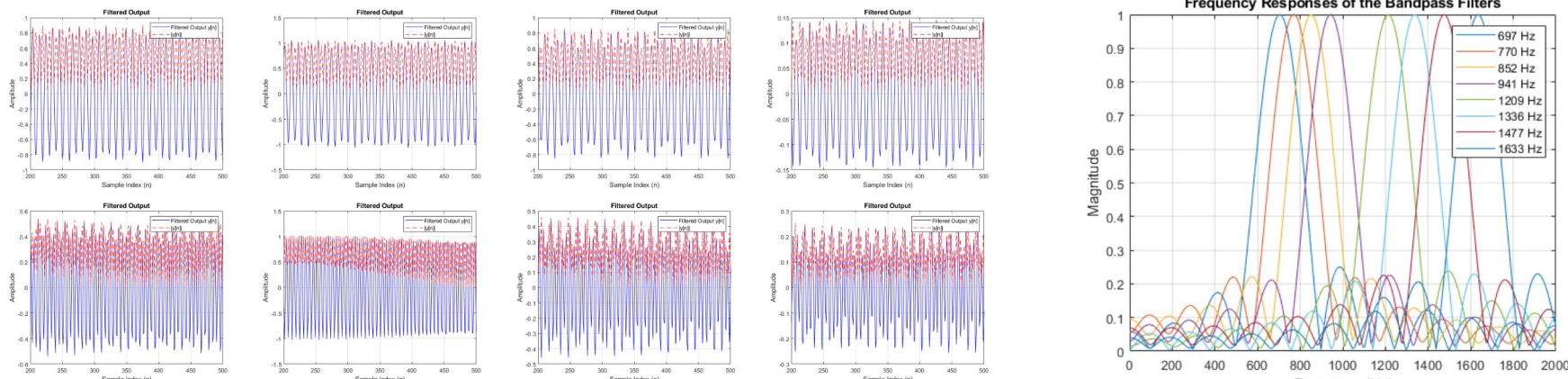
If the maximum amplitude is greater than or equal to 0.59, the score (`sc`) is set to 1, indicating the presence of that frequency in the signal

If the maximum amplitude is less than 0.59, the score is 0, indicating the absence of that frequency

Conclusion

The result of the `dtmfscore` function is a binary score (1 or 0) indicating whether a specific DTMF tone is present in the input signal segment

This score is used to decode the DTMF signal by identifying which of the 8 frequencies are present



Decode Function

The purpose of "dtmfrun.m" is to decode the DTMF tones to the respective keys.

To achieve this, the "dtmfdesign.m" is required to generate a 8 frequency bandpass filter.

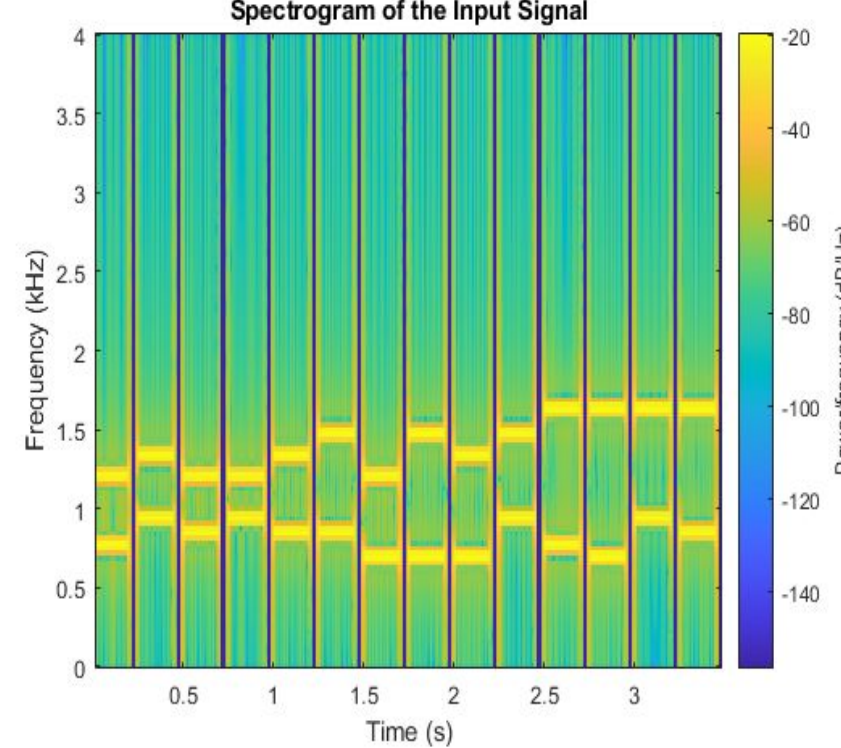
"dtmfcut.m" is a predefined matlab function which is typically used in the DTMF decoding system to segment the input DTMF signal into individual sections corresponding to individual key presses.

Process Segmented Signals with "dtmfscore":

- Each tone is passed through 8 bandpass filters corresponding to the DTMF frequencies.
- The peak amplitude of the filtered output is analyzed.
- Frequencies with amplitudes above a threshold (e.g., 0.59) are marked as present.

Map Frequencies to DTMF Keys:

- Identify the row and column frequencies based on the "dtmfscore" output.
- Use the DTMF keypad frequency table to map the frequency pair to a specific key.
- Example:
 - Detected frequencies: 770 Hz (row) and 1336 Hz (column).
 - Corresponding key: 5.



This features:

- Dynamic processing to identify valid tones.
- Handles noise-free signals with high accuracy.
- Decoded sequence example: 407*89132#BADC.

Telephone Numbers

Objective:

- Verify the functionality of the DTMF system by decoding a sequence of tones generated for a telephone number.

Procedure:

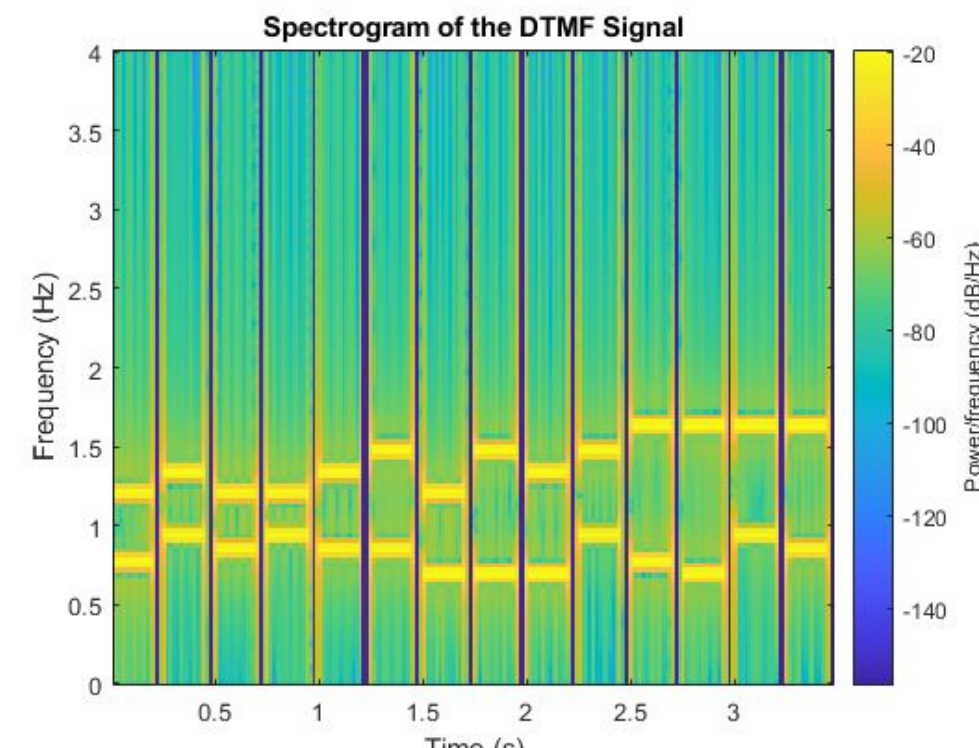
- Generate a DTMF signal for a phone number using "dtmfial.m".
- Decode the signal with "dtmfrun.m".
- Validate the output against the original input.

Example Test Case:

- Input Telephone Number: 407*89132#BADC
- Decoded Output: 407*89132#BADC
- Ensure all keys, including special characters, are correctly identified.

Visualization:

- A spectrogram for the example test case is included to highlight the dual-tone frequency pairs for each key.
- Analyze tone bursts and gaps between digits.



Low Pass Filter

Filter Design Specifications

Cutoff Frequency (ω_c): $0.4\pi = 2 \cdot 2000/10000$ Hz

Sampling Frequency (f_s): 10,000 Hz.

Filter Order (MMM):

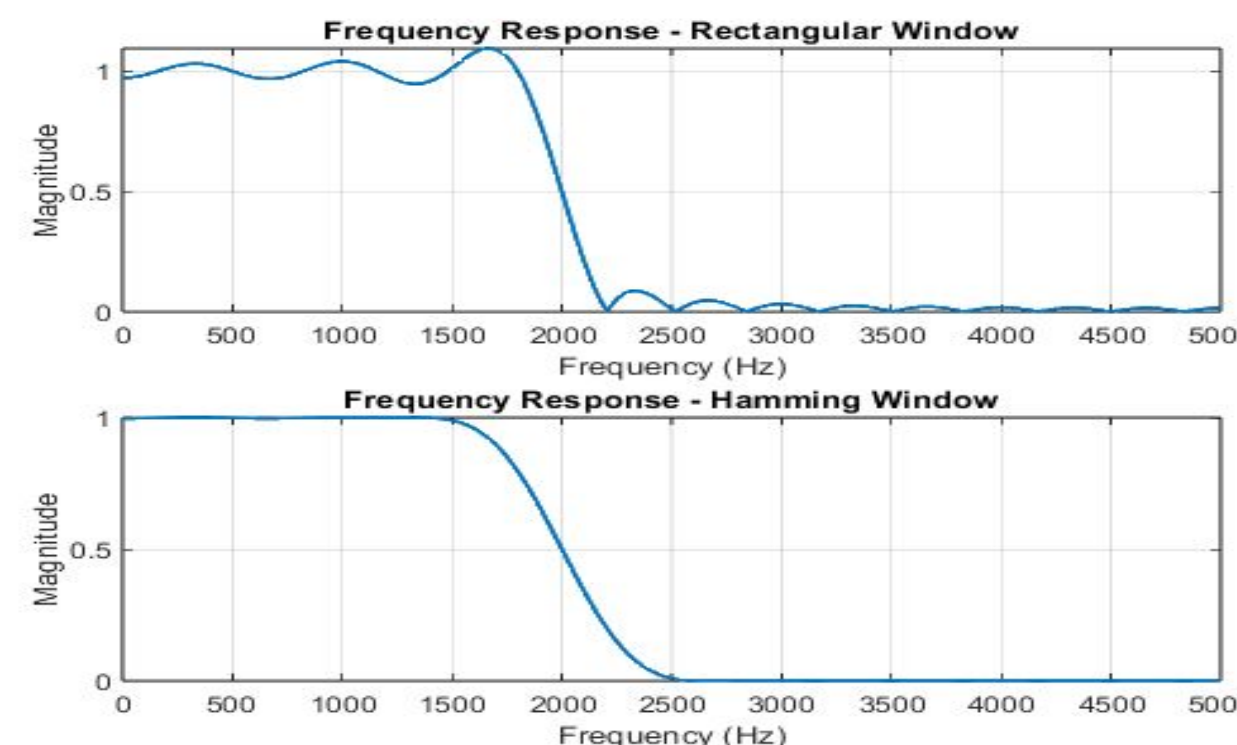
M=30M = 30M=30: Standard Design.

M=60M = 60M=60: For Transition Zone Analysis.

Window Types:

Rectangular Window: Simple truncation with the wider transition zone. It has higher ripple levels in the passband and the stopband.

Hamming Window: Smooth tapering the narrower transition zone. Compared to the rectangular windows, the ripples are smaller due to the tapering.



Transition Zone of the LPF

Transition width ($\Delta\omega = \omega_s - \omega_p$):

The transition zone of a lowpass FIR filter is the frequency range between the passband edge (ω_p) and the stopband edge (ω_s), where the filter transitions from passing to attenuating frequencies. Its width ($\Delta\omega = \omega_s - \omega_p$) reflects the sharpness of the filter, with narrower zones indicating closer approximation to an ideal filter. Achieving a smaller transition zone requires higher filter orders or advanced windows like the Hamming window, which balance smooth attenuation and minimal ripples.

Observations:

Increasing M reduces $\Delta\omega$

Relation: $\Delta\omega = C/L$, where $L = M+1$ (M: filter order)

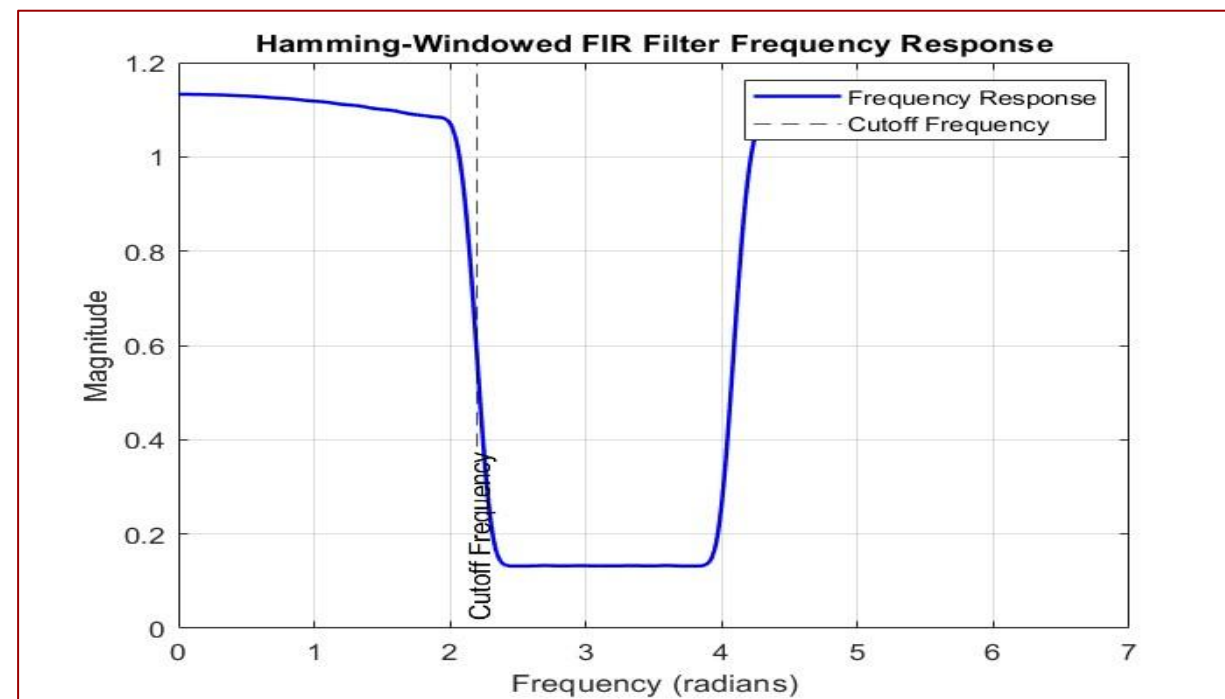
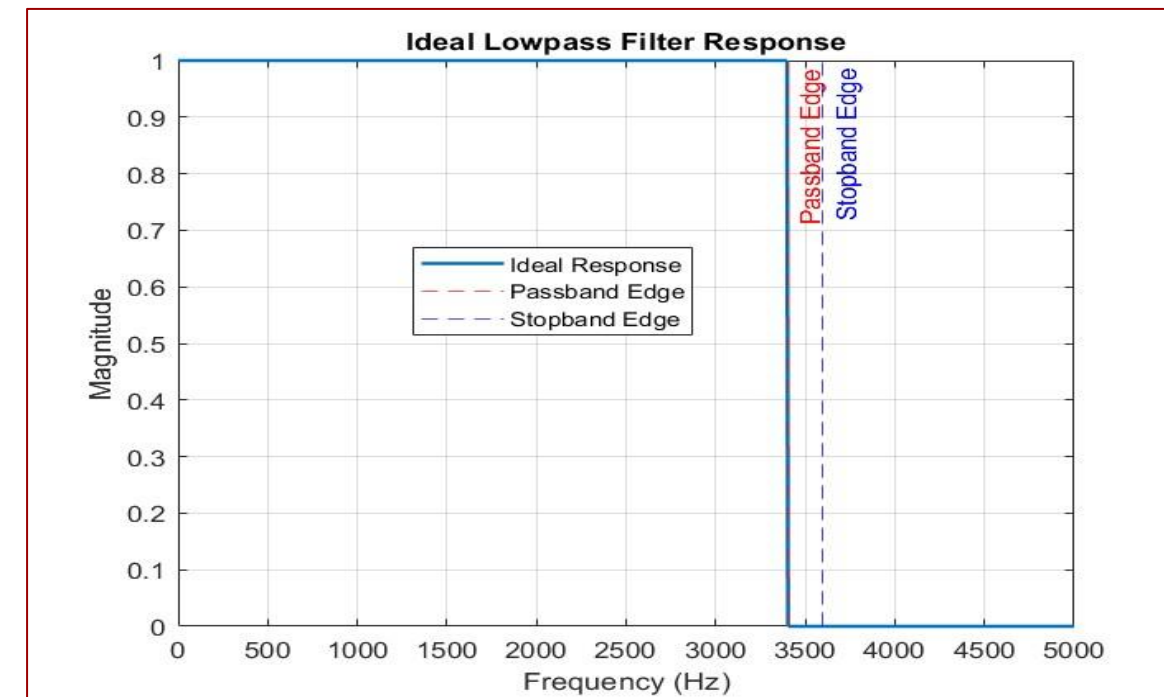
Filter design meeting specifications

$\omega_p = 0.68\pi$ (Passband edge)
 $\omega_s = 0.72\pi$ (Stopband edge)
 $\delta_p = 0.01$ (Passband ripple tolerance)
 $\delta_s = 0.01$ (Stopband ripple tolerance)

$$M = \frac{-20 \log_{10}(\delta_s)}{6.6 \times (\omega_s - \omega_p)}$$

Predicted Filter Order (M): 49
Predicted Filter Length (L): 50
Chosen Cutoff Frequency (ω_c): 2.1991 radians/sample
Passband Ripple: 0.1753 (Expected ≤ 0.0100)
Stopband Attenuation: 1.1326 (Expected ≤ 0.0100)
The filter does NOT meet the specifications.

The Filter does not meet the specs as the predicted filter order using the formula is too low for the given specifications.

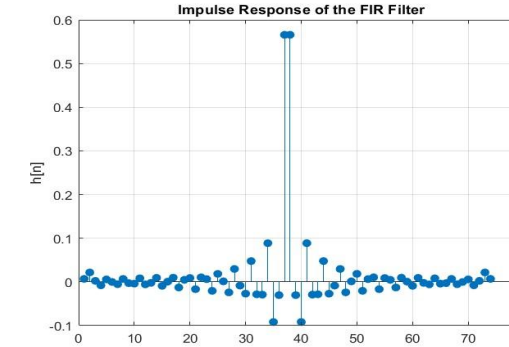


Filter Design using firpm and firpmord

The goal is to design an FIR low pass filter that meets the given specification for passband and stopband ripple using the MATLAB functions **firpmord** and **firpm**.

$\omega_p = 0.68\pi$ (Passband edge)
 $\omega_s = 0.72\pi$ (Stopband edge)
 $\delta_p = 0.05$ (Passband ripple tolerance)
 $\delta_s = 0.01$ (Stopband ripple tolerance)
 ω_p, ω_s are normalized for nyquist frequency

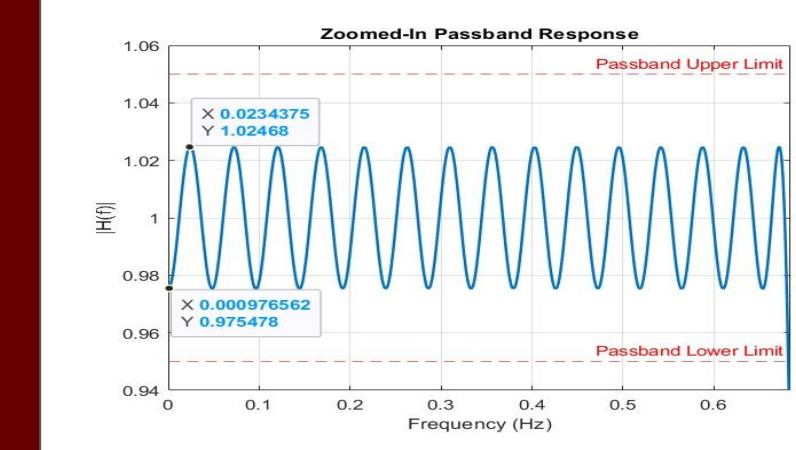
Passband Weight: 1.00
Stopband Weight: 5.00
Estimated Filter Order: 73
Band Edges (Normalized): [0.0000, 0.6800]
Band Edges (Normalized): [0.7200, 1.0000]
Desired Amplitudes: [1.0000, 0.0000]
Weights: [1.0000, 5.0000]



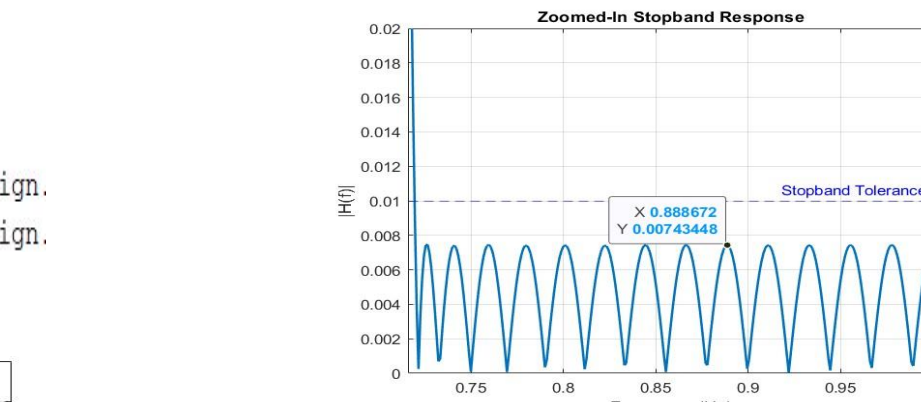
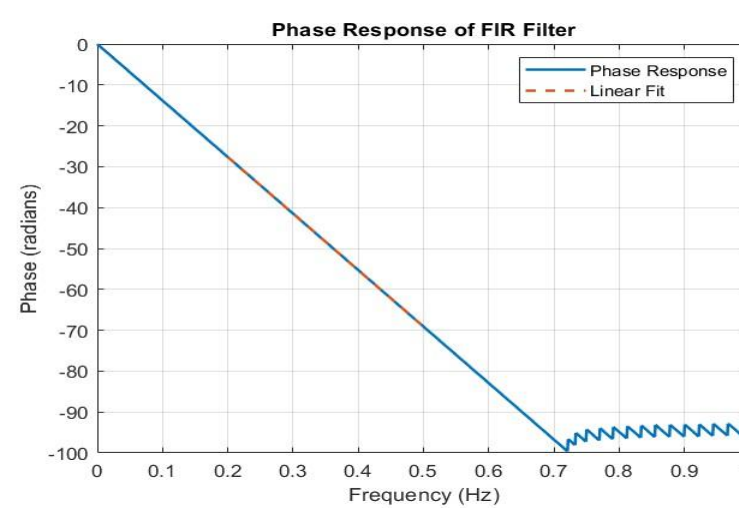
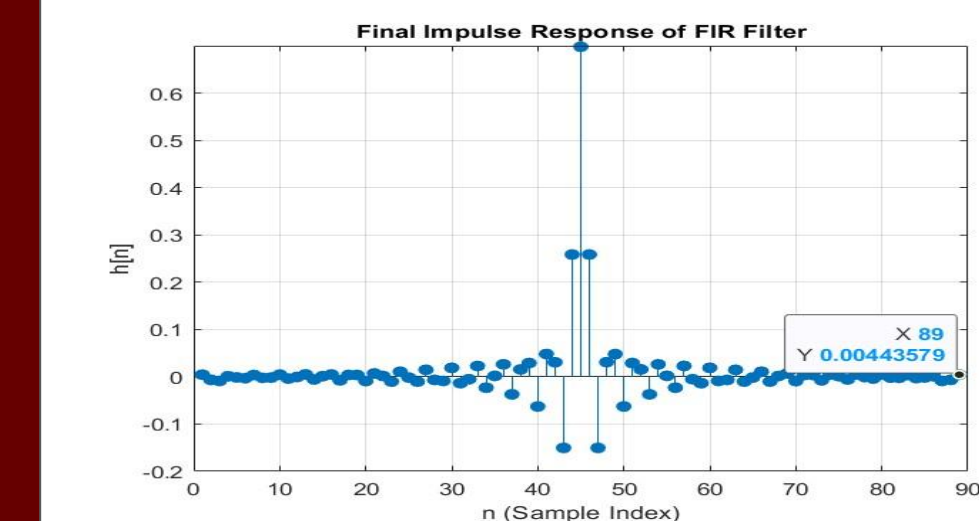
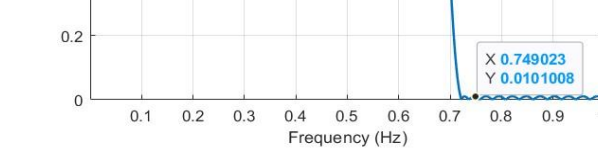
Use the outputs of firpmord as inputs to the function firpm to run the optimization and obtain the FIR filter

% Use firpmord to determine filter order and specifications
[n, fo, ao, w] = firpmord([fp, fs], [1, 0], [delta_p, delta_s], fs);
% Compute Filter Coefficients
b = firpm(n, fo, ao, w); % FIR filter coefficients

The estimated order of the filter does not meet the specifications.
Increasing the order and re-run the firpm for optimizing the filter to meet the specification:



Increased Filter Order: 74
New Passband Ripple: 0.1003 (Expected: 0.0500)
New Stopband Ripple: 0.0101 (Expected: 0.0100)
Passband ripple does NOT meet the specification.
Stopband ripple does NOT meet the specification.
Further Increased Filter Order: 88
New Passband Ripple: 0.0497 (Expected: 0.0500)
New Stopband Ripple: 0.0075 (Expected: 0.0100)
Passband ripple meets the specification after further redesign.
Stopband ripple meets the specification after further redesign.



Slope of Linear Phase: -138.2301 radians/Hz
The FIR filter has a linear phase.